БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ФАКУЛЬТЕТ ПРИКЛАДНОЙ МАТЕМАТИКИ и ИНФОРМАТИКИ
Кафедра многопроцессорных сетей и систем

Рафеенко Е.Д.

Программирование распределенных систем, Web- программирование

Локализация и интернационализация

Содержание

- Локали
- Локализация данных
- Интернационализация чисел, дат

Интернационализация

- Интернационализация программы (i18n)
  - Написание программы, работающий в различных языковых окружениях
- Локализация программы (l10n)
  - Адаптация интернационализированной программы к конкретным языковым окружением
- Пакеты
  - java.util
  - java.text

Локаль

- Идентифицирует используемое языковое окружение
- Класс Locale, пакет java.util

Задание локали

- Использование констант: Locale.US, Locale.FRANCE
- Использование Builder:

```
Locale locale = new Locale.Builder()
  .setLanguage("fr")
  .setRegion("CA")
  .setVariant("POSIX")
  .setScript("Latn")
  .build();
```

- Статический фабричный метод:

```
Locale uk = Locale.forLanguageTag("en-UK");
```

Пример: локализованные данные

- en_UK_windows
  choose the folder containing colour information
- en_UK_unix
  choose the directory containing colour information
- en_US
  choose the folder containing color information
- ru_RU_unix
  Выберите каталог, содержащий цветовую информацию

Методы для Locale

getDefault() возвращает текущую локаль, сконструированную на основе настроек операционной системы.

Составные локали

- **getLanguage()** – код языка региона
- **getDisplayLanguage** – название языка

- **getCountry()** – код региона
- **getDisplayCountry()** – название региона

Получение

- **getAvailableLocales()** – список доступных локалей
- Поля UK, US, FRANCE, …

Locale.***setDefault***(locale) устанавливает текущую локаль

Локализация данных

- Хранение данных, зависящих от локали в одном месте

- Класс ResourceBundle, пакет java.util предназначен для хранения объектов, специфичных для локали

Локализация данных

Набор ресурсов – это фактически набор классов, имеющих одно базовое имя. За базовым именем следует

- код языка;
- код страны;
- признак variant.

имя базового класса: **MyResources**; для английской локали имя класса будет **MyResources_en**, для русской – **MyResources_ru**.

MyResources_de_CH –швейцарский вариант немецкого языка.

MyResources_de_CH_UNIX - для платформы UNIX

Загрузка ресурсов

Загрузка объекта для нужной локали :

```
Locale currentLocale = new Locale.Builder()
  .setLanguage(" de ")
  .setRegion(" CH ");

ResourceBundle messages =
ResourceBundle.getBundle("MyResources", currentLocale);
```

Порядок загрузки ресурсов

список возможных имен ресура генерируется на основе
- указанного базового имени (первый параметр),
- указанной локали (второй параметр),
- локали по умолчанию

указанная локаль имеет более высокий приоритет, чем локаль по умолчанию.

необходимо найти ResourceBundle для локали fr_CH (Швейцарский французский), локаль по умолчанию en_US, название базового класса MyResources, порядок поиска подходящего ResourceBundle:
MyResources_fr_CH
MyResources_fr
MyResources_en_US
MyResources_en
MyResources

Задание ресурсов

• Результат работы getBundle - загрузка необходимого файла ресурсов (".properties") в память.

• Необходимо указывать полностью квалифицированное имя класса ресурсов, т.е. имя пакета, имя класса. Класс ресурсов должен быть доступен в контексте его вызова.

• Всегда должен создаваться базовый файл без суффиксов.

PropertyResouceBundle

В .properties файле информация представляет собой набор строк вида
key1= value1
key2= value2

Например, файл text_en_US.properties:
str1 = To be or not to be
str2 = This is a question

файл text_bel_BY.properties:
str1 = Быць або не быць
str2 = Вось у чым пытанне

файл text.properties:
str1 = Быть или не быть
str2 = Вот в чем вопрос

Локализация данных

Класс ResourceBundle помещают в пакет resources,
файл свойств – в каталог resources.
Чтобы инстанциировать нужный класс, необходимо указать полный путь к этому классу (файлу):
getBundle("resources.MyResource", Locale.getDefault());

Используемые методы:
Enumeration <String> getKeys(String key) – получить перечисление всех ключей
String getString(String key) – извлекает значение по ключу

Локализация данных

```
ResourceBundle bundle =

ResourceBundle.getBundle("resources.UsageResourceBundle");

System.out.println("str1="+bundle.getString("str1"));
System.out.println("str1="+bundle.getString("str2"));
```

```
str1=Usage:
str1= class
```

Локализация данных

```
public class International {
public static void main(String[] args) {
String country = "", language = "";
System.out.println("1 - Английский");
System.out.println("2 - Белорусский");
System.out.println("Любой символ - Русский");
char i = 0;
try {
i = (char) System.in.read();
} catch (IOException e1) {
e1.printStackTrace();}
switch (i) {
case '1':
country = "US";
language = "EN";
break;
case '2':
country = "BY";
language = "BEL";}
Locale currentLocale = new Locale(language, country);
```

Локализация данных

```
ResourceBundle rb =
ResourceBundle.getBundle("resources.text", currentLocale);
try {
System.out.println(new String(
  rb.getString("str1").getBytes("ISO-8859-1"), "CP1251"));
System.out.println(new String(
rb.getString("str2").getBytes("ISO-8859-1"), "CP1251"));
} catch (UnsupportedEncodingException e) {
e.printStackTrace();
}
}}
```

Интернационализация чисел, дат

- Вывод данных в соответствии с языковым контекстом
- Типы данных
  - Числа
  - Время и дата
  - Сообщения
- Пакет java.text DateFormat, NumberFormat

Стандартные форматировщики чисел

- Класс NumberFormat
- Получение форматировщиков чисел
- Методы
  - getInstance(locale)
  - getNumberInstance(locale) – обычные числа
  - getIntegerIntance(locale) – целые числа (с округлением)
  - getPercentInstance(locale) – проценты
  - getCurrencyInstance(locale) – валюта

Интернационализация чисел

- Методы
  - String format(long) – форматировать целое число
  - String format(double) – форматировать число с плавающей точкой
  - Number parse(String) – разобрать локализованное число
- Исключение
  - ParseException – ошибка разбора

Интернационализация чисел

Форматирование числа для текущей локали:

```
myString = NumberFormat.getInstance().format(myNumber);
```

Форматирование числа для заданной локали:

```
NumberFormat nf =
NumberFormat.getInstance(Locale.FRENCH).format(myNumber)
;
```

Интернационализация дат

- Класс DateFormat
- Получение форматировщиков времени и дат
- Методы
  - getDateInstance([dateStyle[, locale]]) – даты
  - getTimeIntance([timeStyle[, locale]]) – времени
  - getDateTimeIntance([dateStyle, timeStyle, [locale]]) – даты и времени
- Стили
  - DEFAULT, FULL, LONG, MEDIUM, SHORT

Интернационализация дат

Методы

- String format(date) – форматировать дату/время
- Date parse(String) – разобрать локализованную дату/время

- Исключение
  - ParseException – ошибка разбора

Интернационализация дат

Пример

```
DateFormat df =
DateFormat.getDateInstance(DateFormat.LONG,
new Locale.Builder()
.setLanguage("ru").setRegion("RU").build() );
String dat = df.format(new Date ());
```

Thymeleaf - messages

По умолчанию приложение использует Standard Message Resolver, реализованный с помощью org.thymeleaf.messageresolver.StandardMessageResolver.

Стандартный message resolver ожидает найти сообщения для /WEB-INF/templates/**home**.html в файлах свойств в той же папке и с тем же именем, что и у шаблона.

- /WEB-INF/templates/home_en.properties
- /WEB-INF/templates/home_кг.properties
- /WEB-INF/templates/home.properties